

POWERTRACK

Sistema de Gestión para Gimnasio

Ejercicio Resuelto — Etapas del Desarrollo de Software (SDLC)



Tema

SDLC Completo



Sistema

POWERTRACK



Estado

Resuelto

Lenguaje claro y sencillo para todos

Parte 1 — Conceptos Básicos

¿Qué es el SDLC y por qué importa?

Pregunta 1 — ¿Qué es el SDLC?

Es como una "receta de cocina" para crear software. Son los pasos ordenados que sigue un equipo para construir un programa desde cero: planear, diseñar, programar, probar y lanzarlo, asegurando calidad y sin salirse del presupuesto.

Pregunta 3 — ¿Por qué definir bien el alcance?

Para no terminar construyendo algo diferente a lo que el cliente quería. Evita gastos extra, pérdida de tiempo y frustraciones. Si no defines el alcance, el proyecto puede crecer sin control y nunca terminar.

Pregunta 2 — Las 7 Fases del SDLC

1
Planificación

2
Análisis

3
Diseño

4
Programación

5
Pruebas

6
Lanzamiento

7
Mantenimiento

Parte 2 — Tipos de Requisitos

¿Qué HACE el sistema vs. qué tan BIEN lo hace?

Funcional = el sistema HACE algo específico

a) FUNCIONAL ✓

Iniciar sesión con correo y contraseña.

→ Describe una acción concreta que el sistema realiza.

c) FUNCIONAL ✓

Registrar pagos mensuales.

→ Es una función específica que el sistema ejecuta.

No Funcional = CÓMO debe comportarse

b) NO FUNCIONAL ⚡

Responder en menos de 2 segundos.

→ Define qué tan rápido debe ser, no qué hace.

d) NO FUNCIONAL ⚡

Soportar 500 usuarios al mismo tiempo.

→ Es un requisito de capacidad, no una función.

Parte 3 — Diseño del Sistema

Preguntas 5 y 6

Pregunta 5 — ¿Diferencia entre HLD y LLD?

HLD — Diseño General	LLD — Diseño Detallado
Es el plano general de la casa: qué cuartos tendrá, dónde va la cocina, cómo se conectan los espacios. En POWERTRACK: define que habrá un módulo de pagos, uno de usuarios y uno de rutinas, y cómo se comunican entre sí.	Es el plano con medidas exactas: dónde va cada enchufe, el ancho de cada puerta. En POWERTRACK: define la clase Pago con sus campos (monto, fecha, estado) y el método registrar() paso a paso.

Pregunta 6 — ¿Qué patrón separa el sistema en Modelo, Vista y Controlador?

El patrón MVC. El Modelo maneja los datos, la Vista es lo que ve el usuario en pantalla, y el Controlador es el "cerebro" que conecta ambos. Así el código es más ordenado y fácil de mantener.

Partes 4 y 5 — Código y Pruebas

Preguntas 7 y 10

Pregunta 7 — ¿Qué es Clean Code?

Es escribir código que cualquier otro programador pueda leer y entender fácilmente, como si fuera un texto bien redactado.

Nombres claros

Variables y funciones con nombres que expliquen qué hacen.

Funciones cortas

Cada función hace una sola cosa y la hace bien.

Sin repeticiones

No copiar y pegar código; reutilizar lo que ya existe.

Comentarios útiles

Solo comentar lo que no es obvio, no todo.

Pregunta 10 — ¿Qué pruebas validan funciones o métodos individuales?

Las Pruebas Unitarias (Unit Tests). Son las pruebas más pequeñas posibles: verifican que cada función del código funcione bien por separado, como probar cada ingrediente antes de cocinar el plato completo.

Ejemplo en POWERTRACK: probar que el método `calcularTotal()` devuelve el monto correcto antes de conectarlo con el resto del sistema.

Parte 6 — Despliegue y Mantenimiento

Preguntas 12, 13 y 14

Pregunta 12 — ¿Qué significa CI/CD?

CI/CD = Integración Continua / Entrega Continua. Es un proceso automático que revisa el código cada vez que alguien hace un cambio y lo publica en el sistema sin hacerlo manualmente. Ahorra tiempo y evita errores humanos. Como una línea de ensamblaje automática.

Pregunta 13 — ¿Qué herramienta empaqueta aplicaciones en contenedores?

Docker. Permite empaquetar una aplicación con todo lo que necesita para funcionar (como meterla en una caja lista para usar). Así funciona igual en cualquier computador, sin el típico problema de "en mi máquina sí funciona".

Pregunta 14 — Mantenimiento Correctivo vs Evolutivo

Correctivo

Arreglar algo que está roto o que falla. Se activa cuando hay un bug o error reportado por usuarios.

Evolutivo

Agregar mejoras o nuevas funciones porque el negocio creció o cambió. No es un error, es evolución.

Pregunta Final — Análisis

App bancaria que no soporta muchos usuarios

"Una aplicación bancaria falla porque no soporta muchos usuarios conectados al mismo tiempo."

¿Qué tipo de requisito falló?

Falló un Requisito No Funcional, específicamente de escalabilidad y rendimiento. El sistema no fue diseñado para aguantar una gran cantidad de usuarios simultáneos, lo cual en una app bancaria es crítico.

¿En qué fase del SDLC debió detectarse?

En la fase de Análisis de Requisitos. Es el momento donde se debe preguntar: ¿cuántos usuarios usarán el sistema? ¿en horarios pico? Si ese requisito se hubiera definido desde el inicio, el equipo habría diseñado una arquitectura capaz de soportar la carga.

Lección: Los requisitos no funcionales son tan importantes como los funcionales — ignorarlos sale muy caro.

Resumen General

1 SDLC

Receta ordenada para crear software de calidad.

2 Requisitos

Funcional = qué hace. No funcional = qué tan bien.

3 Diseño HLD/LLD

Plano general vs. plano detallado del sistema.

4 MVC

Patrón que separa datos, pantalla y lógica.

5 Clean Code

Código legible, claro y sin repeticiones.

6 Pruebas Unitarias

Prueban cada función por separado.

7 CI/CD

Publicación automática y sin errores humanos.

8 Docker

Empaca la app en una caja lista para cualquier máquina.

9 Mantenimiento

Correctivo (arreglar) vs. Evolutivo (mejorar).